Math for Data Science - Lab 6

Course: Math for Data Science

We open with a warm-up on paper: computing a **maximum likelihood estimate** by hand. The rest of this lab is about what happens to averages when you take a lot of them. We start with **covariance and correlation** - how two random variables move together, and what correlation does and does not capture. Then the two **limit theorems** that the rest of statistics rests on: the **law of large numbers**, which says the sample mean converges to the truth, and the **central limit theorem**, which says how it is distributed on the way there - first used as an approximation tool, then demonstrated by simulation. We finish with **Monte Carlo integration**, which is the law of large numbers put to work.

Exercises 1-4 are the core of the lab; 5 and 6 are optional. Questions 2 and 3 of exercise 2 are marked *[optional]* too, and can be skipped if you are short of time.

Every question is marked with what kind of answer it wants:

- **[Code]** - the code is written for you except for one or two lines, marked `# here your code`. The comment just above the gap names the function to use. Fill it in, run the cell, and play with it. None of it asks you to write a program from scratch.
- **[Pen & paper]** - a calculation, derivation or explanation. Write it in the cell marked *here your solution* (or on paper).
- **[Code + Pen & paper]** - fill in the code, then explain what you see in the text cell below.

Setup

Run the cell below once to import everything we need.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
```

Using R instead of Python (optional)

The lab is written in Python, but you can write your answers in R inside this same notebook. This needs R 4.5 or newer, and Jupyter started with rpy2 added: `uv run --with rpy2 jupyter lab`. The `README.md` of the course folder has the details and what to do when it does not work.

If you want to use R instead of Python, uncomment and run the following cell:

```
# %load_ext rpy2.ipython
```

Then, in any subsequent code cell, add `%%R` as the first line to run that cell as R:

```
%%R
x <- rbinom(1000, size = 10, prob = 0.3)
hist(x)
```

Keep running the given Python cells (imports, data loading) as they are. The hints name Python functions, such as `rng.binomial`; you will need to find the R equivalent (`rbinom`) yourself.

Exercise 1: Maximum Likelihood by Hand

A basketball player wants to know her free-throw percentage p - the probability that a single shot goes in. Every shot is assumed independent of the others, with the same p. She collects data with two different practice routines:

- **Sets:** she shoots a set of 10 free throws and records how many went in. The number of makes in a set is $X \sim \text{Binomial}(10, p)$:

$$P(X = x) = \binom{10}{x} p^x (1-p)^{10-x}, \qquad x = 0, 1, ..., 10$$

- **Streaks:** she shoots until her first miss and records how many she made before it. The length of a streak is $Y \sim$ Geometric, counting makes before the first miss:

$$P(Y = y) = p^y (1-p), \qquad y = 0, 1, 2, ...$$

Over one week she records

routine	data
3 sets of 10	$x_1 = 7, x_2 = 8, x_3 = 6$
4 streaks	$y_1 = 3, y_2 = 7, y_3 = 2, y_4 = 6$

Recall the recipe: write down the likelihood $L(p)$ - the probability of the observed data, read as a function of p - take the logarithm $\ell(p) = \log L(p)$, set $\ell'(p) = 0$ and solve.

1. [Pen & paper] Use only the three sets. Write down $L(p)$ and $\ell(p)$, and derive the MLE $\hat{p}$ in general for n sets of 10. What is its value for this data?

here your solution

2. [Pen & paper] Now use only the four streaks. Write down $\ell(p)$ and derive $\hat{p}$ in general for m streaks. What is its value for this data?

here your solution

3. [Pen & paper] The two routines measure the *same* p, so she should use all the data at once. Write down the joint log-likelihood $\ell(p)$ of the sets *and* the streaks, derive $\hat{p}$ and compute it. Then show that $\ell''(p) < 0$, so it really is a maximum.

> **Hint:** the data sets are independent, so the joint likelihood is the product of the two likelihoods - and the joint log-likelihood is the sum of the two log-likelihoods.

here your solution

4. [Pen & paper] The sets alone gave 0.70 and the streaks alone about 0.82, yet the combined estimate is 0.75, not their average 0.76. Explain why. How much weight does each routine get in the combined estimate?

here your solution

Exercise 2: Covariance and Correlation

The covariance of two random variables measures how they move together:

$$\mathrm{Cov}(X, Y) = E\left[(X - EX)(Y - EY)\right] = E(XY) - E(X)E(Y)$$

It inherits the units of X times the units of Y, which makes its magnitude hard to read. The **correlation** rescales it to a unitless number in $[-1, 1]$:

$$\mathrm{Corr}(X, Y) = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}}$$

1. [Pen & paper] In lab 3 you showed that $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ requires independence. The general formula, which needs nothing, is

$$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X, Y)$$

Use this to explain both of the results you found in lab 3: why additivity holds under independence, and why $\mathrm{Var}(X + X) = 4\ \mathrm{Var}(X)$.

here your solution

2. [Code] *[optional]* Verify the general formula by simulation. Build a pair of *dependent* variables - draw X, then set $Y = 2X + \varepsilon$ with independent noise ε - and check that $\mathrm{Var}(X + Y)$ matches $\mathrm{Var}(X) + \mathrm{Var}(Y) + 2\ \mathrm{Cov}(X, Y)$ while the naive additivity rule does not.

> **Hint:** arithmetic on arrays works element by element, so $Y = 2X + \varepsilon$ can be typed as it stands. `np.cov(X, Y, ddof=0)` returns the 2x2 covariance *matrix*; the covariance you want is the off-diagonal entry `[0, 1]`. (`ddof=0` makes it match `.var()`.)

```
rng = np.random.default_rng(11)

n = 500_000
X = rng.normal(0, 1, size=n)
eps = rng.normal(0, 1, size=n)
```

```
# Two lines: `Y` = 2X + eps, and `cov_xy`, the covariance of X and Y from `np.cov`.
# here your code

print(f"Var(X)              = {X.var():.4f}")
print(f"Var(Y)              = {Y.var():.4f}")
print(f"Cov(X, Y)           = {cov_xy:.4f}")
print()
print(f"Var(X + Y) simulated                    = {(X + Y).var():.4f}")
print(f"Var(X) + Var(Y) + 2Cov(X, Y)               = {X.var() + Y.var() + 2
* cov_xy:.4f}")
print(f"Var(X) + Var(Y)  (naive, wrong here)    = {X.var() + Y.var():.4f}")
print()
print(f"Corr(X, Y)          = {np.corrcoef(X, Y)[0, 1]:.4f}")
```

3. [Code + Pen & paper] *[optional]* Correlation measures **linear** association only, and this is a much sharper limitation than it sounds. Construct a pair of variables that are *perfectly* dependent - Y is an exact, deterministic function of X - but have correlation approximately zero. Plot them. What does this tell you about correlation?

> **Hint:** try $X \sim \text{Unif}(-1, 1)$ - `rng.uniform(-1, 1, size=...)` - and $Y = X^2$. `np.corrcoef(X, Y)` returns the 2x2 correlation matrix; the correlation is entry `[0, 1]`.

```
rng = np.random.default_rng(12)

# Three lines: `X2`, 200,000 uniform draws on [-1, 1]; `Y2` = X2 squared;
# and `corr`, their correlation from `np.corrcoef`.
# here your code

print(f"Corr(X, Y) = {corr:.5f}")

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(6, 4.5))
ax.scatter(X2[:3000], Y2[:3000], s=4, alpha=0.3, color="steelblue")
ax.set_xlabel("$X$")
ax.set_ylabel("$Y = X^2$")
ax.set_title("perfectly dependent, essentially uncorrelated")
fig.tight_layout()
plt.show()
```

here your solution

Exercise 3: The Law of Large Numbers

Let $X_1, X_2, \ldots$ be i.i.d. with mean μ and finite variance, and let $\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$. The **law of large numbers** says that

$$\bar{X}_n \xrightarrow{p} \mu \quad \text{as} \quad n \to \infty$$

> **Notation:** the p over the arrow stands for convergence **in probability** - for every $\varepsilon > 0$, $P(|\bar{X}_n - \mu| > \varepsilon) \to 0$ as $n \to \infty$.

The sample mean converges to the true mean. This is the theorem that licenses every simulation you have run in this course.

1. [Code] Flip a fair coin 100,000 times and plot the running proportion of heads against the number of flips. Do this for six independent sequences on the same axes, and use a log scale for n.

> **Hint:** `np.cumsum(tosses)` gives the running count of heads (1 = heads, 0 = tails). Divide it by `flips` - the number of flips so far - to get the running proportion.

```
rng = np.random.default_rng(1713)

n_flips = 100_000
flips = np.arange(1, n_flips + 1)

fig, ax = plt.subplots(figsize=(8, 4.5))
for _ in range(6):
    tosses = rng.integers(0, 2, size=n_flips)
    # The running proportion of heads after each flip, using `np.cumsum`.
    # here your code
    ax.plot(flips, running, linewidth=1)

# --- plotting, written for you ---
ax.axhline(0.5, color="red", linestyle="--", linewidth=2, label="$\\mu = 0.5$")
ax.set_xscale("log")
ax.set_ylim(0, 1)
ax.set_xlabel("Number of flips (log scale)")
ax.set_ylabel("Running proportion of heads")
ax.legend()
fig.tight_layout()
plt.show()
```

2. [Code + Pen & paper] A gambler has just seen ten heads in a row. He argues that tails is now "due", because the law of large numbers guarantees the proportion must even out to one half.

Where exactly does the argument go wrong? Settle it by simulation: take sequences that begin with ten heads, and check whether the flips that follow are biased towards tails.

> **Hint:** `rng.integers(0, 2, size=(rows, columns))` draws a whole table of coin flips at once: one row per sequence. `after.sum(axis=1)` counts the heads in each row.

```
rng = np.random.default_rng(1954)

# 200,000 sequences, each of 10 forced heads followed by 1,000 fair flips
n_seq, n_after = 200_000, 1_000

# The flips *after* the ten heads: a table of 0/1 with n_seq rows and n_after
columns.
# here your code

print(f"P(heads) on the very next flip      = {after[:, 0].mean():.4f}")
print(f"P(heads) over the next 1,000 flips  = {after.mean():.4f}")

# The overall proportion of heads in each sequence, *including* the 10 forced
heads:
# (10 + heads after) / (10 + n_after).
# here your code
print(f"\nmean overall proportion after 1,010 flips = {overall.mean():.4f}")
```

here your solution

Exercise 4: Using the CLT as an Approximation

Suppose we have a computer program consisting of $n = 100$ pages. Let X_i be the number of errors in the i-th page of code. Suppose that the X_i are Poisson with mean 1 and that they are independent. Let $Y = \sum_{i=1}^{n} X_i$ be the total number of errors in the document.

1. [Pen & paper] Use the central limit theorem to approximate $P(Y < 90)$.

> **Reminder:** the central limit theorem says that for i.i.d. $X_1, \ldots, X_n$ with mean μ and variance σ^2, the standardized sample mean is approximately standard normal for large n:
>
> $$\frac{\sqrt{n}\left(\bar{X}_n - \mu\right)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1)$$
>
> **Notation:** the d over the arrow stands for convergence **in distribution** - the CDF of the left-hand side approaches the CDF of the right-hand side.

here your solution

2. [Code] Check the approximation by simulation: generate many replications of the total error count Y and compare the simulated $P(Y < 90)$ to both the CLT approximation and the exact answer.

> **Hint:** a sum of independent Poisson random variables is itself Poisson, with the rates added - so $Y \sim \text{Pois}(100)$ exactly. That gives you a ground truth to compare against. The functions you need: `rng.poisson(lam=1.0, size=(rows, 100))` with `.sum(axis=1)` for the simulation, `np.mean(Y < 90)` for a simulated probability, `stats.norm.cdf(z)` for $\Phi(z)$, and `stats.poisson.cdf(k, mu=100)` for the exact $P(Y \leq k)$.

```
rng = np.random.default_rng(2025)

# Four lines:
#   `Y`         - 200,000 simulated programs: a (200_000, 100) table of Pois(1)
errors, summed per row
#   `simulated` - the proportion of `Y` below 90
#   `clt`       - the CLT approximation Phi(-1)
#   `exact`     - P(Y < 90) = P(Y <= 89) for Y ~ Poisson(100)
# here your code

print(f"simulated   P(Y < 90) = {simulated:.4f}")
print(f"CLT approx  P(Y < 90) = {clt:.4f}")
print(f"exact       P(Y < 90) = {exact:.4f}")
```

3. [Code] That gap has a standard fix. When you approximate a *discrete* distribution by a continuous one, the event $\{Y \leq 89\}$ is better matched by the normal's $\{Y \leq 89.5\}$ - you give each integer its full unit of width. This is the **continuity correction**.

Redo the approximation with a cutoff of 89.5 instead of 90, and compare the two errors.

> **Hint:** Y has mean 100 and standard deviation 10, so use `stats.norm.cdf(cutoff, loc=100, scale=10)`.

```
exact = stats.poisson.cdf(89, mu=100)

# Two lines: `plain`, the normal approximation with cutoff 90, and `corrected`,
# the same with cutoff 89.5.
# here your code

print(f"exact                    = {exact:.5f}")
```

```
print(f"normal, cutoff 90              = {plain:.5f}    (error {abs(plain -
exact):.5f})")
print(f"normal, cutoff 89.5          = {corrected:.5f}    (error {abs(corrected
- exact):.5f})")
```

4. [Code] The same idea gives the **normal approximation to the binomial**. If $X \sim \mathrm{Bin}(n, p)$ then for large n,

$$X \dot{\sim} \mathcal{N}\left(np, np(1-p)\right)$$

Take $X \sim \mathrm{Bin}(100, 0.5)$ and compute $P(X \leq 45)$ exactly, by the normal approximation, and by the normal approximation with a continuity correction. Then plot the binomial PMF with the approximating normal density on top.

> **Hint:** `stats.binom.cdf(k, n, p)` is the exact $P(X \leq k)$. For the normal, use `stats.norm.cdf(cutoff, loc=mean_bin, scale=sd_bin)` - `scale` is the standard deviation, so take `np.sqrt` of the variance.

```
n_bin, p_bin = 100, 0.5

# Five lines: the mean `mean_bin` = np and standard deviation `sd_bin` =
sqrt(np(1-p)),
# then `exact`, `plain` (cutoff 45) and `corrected` (cutoff 45.5).
# here your code

print(f"exact  P(X <= 45)            = {exact:.5f}")
print(f"normal, no correction           = {plain:.5f}    (error {abs(plain -
exact):.5f})")
print(f"normal, with correction      = {corrected:.5f}    (error {abs(corrected
- exact):.5f})")

# --- plotting, written for you ---
k = np.arange(30, 71)
grid = np.linspace(30, 70, 400)

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(k, stats.binom.pmf(k, n_bin, p_bin), color="steelblue", alpha=0.6,
       label="Bin(100, 0.5) PMF")
ax.plot(grid, stats.norm.pdf(grid, loc=mean_bin, scale=sd_bin), color="red",
        linewidth=2, label="$\\mathcal{N}(50, 25)$ density")
ax.set_xlabel("$k$")
ax.set_ylabel("probability / density")
ax.legend()
```

```
fig.tight_layout()
plt.show()
```

Exercise 5: The Central Limit Theorem

[optional]

The LLN says $\bar{X}_n$ lands on μ. The **central limit theorem** says how it is distributed around μ on the way: for i.i.d. X_i with mean μ and variance σ^2,

$$\frac{\sqrt{n}\left(\bar{X}_n - \mu\right)}{\sigma} \xrightarrow{d} \mathcal{N}(0, 1)$$

The remarkable part is what is *missing* from that statement: any assumption about the distribution of the X_i. It can be discrete, skewed, bimodal, anything with a finite variance - the sample mean is approximately normal for large n regardless.

1. [Code] Demonstrate this. For each of four starting distributions - $\text{Unif}(0, 1)$, $\text{Expo}(1)$, $\text{Bern}(0.1)$ and $\text{Pois}(2)$ - and each of $n = 1, 2, 10, 100$, draw 10,000 values of $\bar{X}_n$ and histogram them. The result is laid out as a 4x4 grid, rows by distribution and columns by n.

> **Hint:** `draw((n_reps, n))` returns a table with `n_reps` rows of `n` draws each. Taking `.mean(axis=1)` averages each row, giving 10,000 independent sample means in one go.

```
rng = np.random.default_rng(1733)

n_reps = 10_000
sample_sizes = (1, 2, 10, 100)

# each entry: a name, and a function that draws a table of the given shape
samplers = [
    ("Unif(0, 1)",  lambda shape: rng.uniform(0, 1, size=shape)),
    ("Expo(1)",     lambda shape: rng.exponential(1.0, size=shape)),
    ("Bern(0.1)",   lambda shape: (rng.random(shape) < 0.1).astype(float)),
    ("Pois(2)",     lambda shape: rng.poisson(2.0, size=shape).astype(float)),
]

fig, axes = plt.subplots(len(samplers), len(sample_sizes),
                         figsize=(11, 9))

for row, (name, draw) in enumerate(samplers):
    for col, n in enumerate(sample_sizes):
      # `xbar`: 10,000 sample means of size n, using `draw` and `.mean(axis=1)`.
        # here your code
```

```
        # --- plotting, written for you ---
        ax = axes[row, col]
        ax.hist(xbar, bins=40, density=True, color="steelblue", alpha=0.7)
        ax.set_yticks([])
        if row == 0:
            ax.set_title(f"$n = {n}$")
        if col == 0:
            ax.set_ylabel(name, fontsize=10)

fig.suptitle("Distribution of $\\bar{X}_n$ for four starting distributions")
fig.tight_layout()
plt.show()
```

2. [Pen & paper] Look along each row. Which starting distribution needs the largest n before $\bar{X}_n$ looks normal, and why? What does the $n = 1$ column show?

here your solution

3. [Code] Make the convergence quantitative rather than visual. For Expo(1), standardize $\bar{X}_n$ properly - subtract μ and divide by $\sigma/\sqrt{n}$ - and overlay the standard normal density for $n = 1, 5, 30, 200$.

> **Reminder:** for Expo($\lambda = 1$), $\mu = 1$ and $\sigma = 1$.

> **Hint:** the sample means are computed as in question 1: draw a table with `rng.exponential(1.0, size=(50_000, n))` and average each row with `.mean(axis=1)`. For the standardization, `np.sqrt(n)` is $\sqrt{n}$.

```
rng = np.random.default_rng(1810)

mu, sigma = 1.0, 1.0
grid = np.linspace(-4, 4, 400)

fig, axes = plt.subplots(1, 4, figsize=(13, 3.4), sharey=True)
for ax, n in zip(axes, (1, 5, 30, 200)):
    # Two lines: `xbar`, 50,000 sample means of n Expo(1) draws, and `z`, the
    # standardized means (xbar - mu) / (sigma / sqrt(n)).
    # here your code

    # --- plotting, written for you ---
       ax.hist(z,  bins=60,  range=(-4,  4),  density=True,  color="steelblue",
```

```
alpha=0.6)
    ax.plot(grid, stats.norm.pdf(grid), color="red", linewidth=2)
    ax.set_title(f"$n = {n}$")
    ax.set_xlabel("$z$")
axes[0].set_ylabel("density")
fig.tight_layout()
plt.show()
```

Exercise 6: Monte Carlo Integration

[optional]

The law of large numbers is what makes **Monte Carlo integration** work. In lab 5 you estimated the integral of a complicated function $f \geq 0$ by throwing darts: draw i.i.d. points $(X_1, Y_1), \ldots, (X_n, Y_n)$ uniformly in a rectangle $a \leq x \leq b$, $0 \leq y \leq c$ that contains the graph of f, let $I_j = 1$ if the j-th point lands below the curve and $I_j = 0$ otherwise, and use

$$\int_a^b f(x)\,dx \approx c(b-a)\,\frac{1}{n}\sum_{j=1}^{n} I_j$$

The I_j are i.i.d. Bernoulli with success probability $p = E(I_j) = \frac{\int_a^b f(x)\,dx}{c(b-a)}$, the share of the rectangle under the curve.

1. [Pen & paper] Using the law of large numbers, show that this estimator converges in probability to $\int_a^b f(x)\,dx$.

here your solution

There is a second, usually better, Monte Carlo method: instead of counting hits below the curve, **average the function**. Let's consider the integral $I = \int_0^1 x^3\,dx$.

2. [Pen & paper] Calculate the exact value of I.

here your solution

3. [Pen & paper] We would like to construct a Monte Carlo estimate of I. Rewrite I in an expression that involves the probability density function of the uniform distribution over $(0, 1)$. Provide an estimator $\hat{I}$ that converges in probability to I.

here your solution

4. [Code] Calculate an estimate of $\hat{I}$ based on $N = 10,000$ observations.

> **Hint:** `rng.uniform(0, 1, size=10_000)` draws the X_i; `np.mean(...)` averages an array.

```
rng = np.random.default_rng(42)

# Two lines: draw 10,000 uniform values `x`, then `I_hat`, the mean of x cubed.
# here your code

print(f"I_hat = {I_hat:.5f}   (exact: 0.25)")
```

5. [Code + Pen & paper] Now calculate the Monte Carlo estimator for different numbers of observations (ranging from 10 to 10,000) and plot your results. What do you observe?

> **Hint:** inside the loop, `x` already holds n uniform draws. The estimate is the same `np.mean(...)` as in question 4.

```
sample_size = np.arange(10, 10_001, 10)

results = pd.DataFrame({"sample_size": sample_size, "estimate": np.nan})

for idx, n in enumerate(sample_size):
    x = rng.uniform(0, 1, size=n)
   # Store the Monte Carlo estimate for this n in `results.loc[idx, "estimate"]`.
    # here your code

# --- plotting, written for you ---
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.scatter(results["sample_size"], results["estimate"], s=6, color="blue")
ax.axhline(0.25, color="red", linewidth=2, linestyle="--")
ax.set_title("Monte Carlo Estimate vs. Sample Size")
ax.set_xlabel("Sample Size")
ax.set_ylabel("Estimate of $E[X^3]$")
fig.tight_layout()
plt.show()
```

here your solution